

Online Chat Server —Requirements

1. Introduction

The task is to implement a classic web-based online chat application with support for:

- user registration and authentication
- public and private chat rooms
- one-to-one personal messaging
- contacts/friends
- file and image sharing
- basic moderation and administration features
- persistent message history

The application should represent a typical “classic web chat” experience, with straightforward navigation, room lists, contact lists, message history, notifications, and online presence indicators.

The system is intended for moderate scale and should support up to 300 simultaneously connected users.

2. Functional Requirements

2.1 User Accounts and Authentication

2.1.1 Registration

The system shall allow self-registration using:

- email
- password
- unique username

2.1.2 Registration Rules

- Email must be unique.
- Username must be unique.
- Username is immutable after registration.

- Email verification is not required.

2.1.3 Authentication

The system shall support:

- sign in with email and password
- sign out (logs out the current browser session only; other active sessions are not affected)
- persistent login across browser close/reopen

2.1.4 Password Management

The system shall support:

- password reset
- password change for logged-in users

No forced periodic password change is required.

Passwords must be stored securely in hashed form.

2.1.5 Account Removal

The system shall provide a “Delete account” action.

If a user deletes their account:

- their account is removed
- only chat rooms owned by that user are deleted
- all messages, files, and images in those deleted rooms are deleted permanently
- membership in other rooms is removed

2.2 User Presence and Sessions

2.2.1 Presence States

The system shall show contact presence using these statuses:

- online
- AFK

- offline

2.2.2 AFK Rule

A user is considered AFK if they have not interacted with any of their open browser tabs for more than 1 minute. If the user is active in at least one tab, they are considered online.

2.2.3 Multi-Tab Support

The application shall work correctly if the same user opens the chat in multiple browser tabs. The following rules apply across tabs:

- If the user is active in at least one tab, they appear as online to others.
- AFK status is set only when all open tabs have been inactive for more than 1 minute.
- A user becomes offline only when all browser tabs are closed/offloaded by browser as inactive.

2.2.4 Active Sessions

The user shall be able to view a list of their active sessions, including browser/IP details, and log out selected sessions. Logging out from the **current browser** invalidates only the session for that browser. Other active sessions remain valid until the user explicitly logs them out from this screen or they expire naturally.

2.3 Contacts / Friends

2.3.1 Friend List

Each user shall have a personal contact/friend list.

2.3.2 Sending Friend Requests

A user shall be able to send a friend request:

- by username
- from the user list in a chat room

A friend request may include optional text.

2.3.3 Friendship Confirmation

Adding a friend requires confirmation by the recipient.

2.3.4 Removing Friends

A user may remove another user from their friend list.

2.3.5 User-to-User Ban

A user may ban another user.

Ban effect:

- the banned user cannot contact the user who banned them in any way
- new personal messaging between them is blocked
- existing personal message history remains visible but becomes read-only/frozen
- friend relationship between the two users is effectively terminated

2.3.6 Personal Messaging Rule

Users may exchange personal messages only if they are friends and neither side has banned the other.

2.4 Chat Rooms

2.4.1 Chat Room Creation

Any registered user may create a chat room.

2.4.2 Chat Room Properties

A chat room shall have:

- name
- description
- visibility: public or private
- owner
- admins
- members
- banned users list

Room names are required to be unique.

2.4.3 Public Rooms

The system shall provide a catalog of public chat rooms showing:

- room name
- description
- current number of members

The catalog shall support simple search.

Public rooms can be joined freely by any authenticated user unless banned.

2.4.4 Private Rooms

Private rooms are not visible in the public catalog.

Users may join a private room only by invitation.

2.4.5 Joining and Leaving Rooms

- Users may freely join public rooms unless banned from that room.
- Users may leave rooms freely.
- The owner cannot leave their own room.
- The owner may only delete the room.

2.4.6 Room Deletion

If a chat room is deleted:

- all messages in the room are deleted permanently
- all files and images in the room are deleted permanently

2.4.7 Owner and Admin Roles

Each room has one owner.

The owner is always an admin and cannot lose admin privileges.

Admins may:

- delete messages in the room
- remove members from the room
- ban members from the room
- view the list of banned users
- view who banned each banned user

- remove users from the ban list
- remove admin status from other admins, except the owner

The owner may:

- do all actions that an admin can do
- remove any admin
- remove any member
- delete the room

2.4.8 Room Ban Rules

If a user is removed from a room by an admin, it is treated as a ban:

- the user is removed from the room
- the user cannot rejoin the room unless removed from the room ban list

If a user loses access to a room:

- they lose access to room messages through the UI
- they lose access to all files and images in that room
- existing files remain stored unless the room itself is deleted

2.4.9 Room Invitations

Users may invite other users to private rooms.

2.5 Messaging

2.5.1 Room and Personal Chat Model

Personal messages shall behave the same way as room messages from the UI and feature perspective.

Conceptually, a personal dialog is a chat with a fixed participant list of two users.

Personal chats support the same message and attachment features as room chats.

Only room owner/admin moderation applies to room chats; personal chats do not have admins.

2.5.2 Message Content

Users shall be able to send messages containing:

- plain text
- multiline text
- emoji
- attachments
- reply/reference to another message

Maximum text size per message: 3 KB.

Message text must support UTF-8.

2.5.3 Message Replies

A user may reply to another message.

The replied-to message shall be visually outlined or quoted in the message UI.

2.5.4 Message Editing

Users shall be able to edit their own messages.

If a message was edited, the UI shall display a gray “edited” indicator similar to common chat applications.

2.5.5 Message Deletion

Messages may be deleted:

- by the message author
- by room admins in room chats

Deleted messages are not required to be recoverable.

2.5.6 Message Ordering and History

Messages shall be stored persistently and displayed in chronological order.

Users shall be able to scroll through very old history using infinite scroll.

Messages sent to an offline user are persisted and delivered when the recipient next opens the application.

2.6 Attachments

2.6.1 Supported Attachments

Users shall be able to send:

- images
- arbitrary file types

2.6.2 Upload Methods

Attachments may be added by:

- explicit upload button
- copy and paste

2.6.3 Attachment Metadata

The system shall preserve the original file name.

The user may add an optional comment to an attachment.

2.6.4 Access Control

Files and images may be downloaded only by current members of the chat room or by authorized participants of the personal chat.

If a user loses access to a room, they must also lose access to the room's files and images.

2.6.5 Persistence of Attachments

If a user uploaded a file and later loses access to the room:

- the file remains stored
- the user can no longer see, download, or manage it

2.7 Notifications

2.7.1 Unread Indicators

If a user has unread messages in:

- a chat room
- a personal dialog

the UI shall show a notification indicator near the corresponding room or contact.

The unread indicator is cleared when the user opens the corresponding chat.

2.7.2 Presence Update Speed

Online/AFK/offline presence updates should appear with low latency.

3. Non-Functional Requirements

3.1 Capacity and Scale

- The server shall support up to 300 simultaneous users.
- A single chat room may contain up to 1000 participants.
- A user may belong to an unlimited number of rooms.
- For sizing assumptions, a typical user may have around 20 rooms and 50 contacts.

3.2 Performance

- After a user sends a message, it should be delivered to recipients within 3 seconds.
- User online status updates should propagate with latency below 2 seconds.
- The application should remain usable when a room contains very large history, including at least 10,000 messages.

3.3 Persistence

- Messages must be persistently stored and remain available for years.
- Chat history loading shall support infinite scrolling for older messages.

3.4 File Storage

- Files shall be stored on the local file system.
- Maximum file size: 20 MB.
- Maximum image size: 3 MB.

3.5 Session Behavior

- No automatic logout due to inactivity is required.
- Login state shall persist across browser close/open.
- The application shall work correctly across multiple tabs for the same user.

3.6 Reliability

The system should preserve consistency of:

- membership
 - room bans
 - file access rights
 - message history
 - admin/owner permissions
-

4. UI Requirements

4.1 General Layout

The application shall provide a typical web chat layout with:

- top menu
- message area in the center
- message input at the bottom
- rooms and contacts list on the side

4.1.1 Side Layout

- Rooms and contacts are displayed on the right.
- After entering a room, the room list becomes compacted in accordion style.
- Room members are shown on the right side with their online statuses.

4.2 Chat Window Behavior

The chat window shall support standard chat behavior:

- automatic scrolling to new messages when the user is already at the bottom
- no forced autoscroll if the user has scrolled up to read older messages
- infinite scroll for older history

4.3 Message Composition

The input area shall support:

- multiline text entry

- emoji in messages
- file/image attachment
- reply to message

4.4 Notifications in UI

Unread messages shall be visually indicated near:

- room names
- contact names

4.5 Admin UI

Administrative actions shall be available from menus and implemented through modal dialogs.

These actions include:

- ban/unban user
- remove member
- manage admins
- view banned users
- delete messages
- delete room

5. Notes and Clarifications

- Email and username are unique.
- Username is immutable.
- Room names are unique.
- Public rooms are discoverable through the public catalog and may be joined by any authenticated user unless banned.
- Private rooms are available only by invitation.
- Personal dialogs are functionally equivalent to chats, but have exactly two participants.

- Existing personal message history remains visible but frozen after a user-to-user ban.
- If a room is deleted, all its messages and attachments are deleted permanently.
- If a user loses access to a room, they lose access to its messages, files, and images.
- Files persist after upload unless the room is deleted, even if the original uploader later loses access.
- The application should resemble a classic web chat rather than a modern social network or collaboration suite.
- A user is considered offline only when they have no open browser tabs with the application.
- If a user has multiple tabs open, presence is determined by the most active tab: online if any tab is active, AFK only if all tabs have been idle for more than 1 minute.
- Sign out logs out the current browser session only. Other active sessions (other browsers or devices) remain valid.
- Messages sent to offline users are persisted and displayed when the recipient next connects.

6. Advanced requirements

If you manage to implement requirements above quickly. Add additional feature: Jabber protocol support. This should include:

1. Users must be able to connect to the server using Jabber client (choose jabber protocol support level you can afford)
2. Servers must be able to support federation i.e. messages between servers (this will require more complicated setup in docker compose)
3. Use jabber library available for your tech stack.

Most advanced possible implementation must include load test for federation of two servers:

1. 50+ clients connected to one server - A, 50+ to another - B
2. Messaging from A to B and back.

Note: with existing library this feature is rather integration than coding, but with AI agent it could be challenging.

Also, you must add to server UI screens specific to Jabber

1. Connection dashboard for admin
2. Federation traffic info/statistics

7. How to submit HW results

- Create public github repository with your project
- Project MUST be buildable and runnable by `docker compose up` in the root repository folder

Appendix A: Wireframes:

```
+-----+
| LOGO                                     [Sign in] [Register] |
+-----+
```

```
+-----+ +-----+
|          SIGN IN          | |          REGISTER          | |
+-----+ +-----+
| Email                    | | Email                    | |
| [_____]                | | [_____]                | |
| Password                 | | Username                 | |
| [_____]                | | [_____]                | |
| [ ] Keep me signed in    | | Password                 | |
|                          | | [_____]                | |
| ( Sign in )              | | Confirm password         | |
|                          | | [_____]                | |
| Forgot password?         | |                          | |
+-----+ | ( Create account ) |
                  +-----+
```

```
+-----+
| Forgot password          | |
+-----+
| Enter your email to reset password | |
| [_____]                | |
```

```

| ( Send reset link ) |
+-----+

+-----+

| ChatLogo | Public Rooms | Private Rooms | Contacts | Sessions | Profile ▼ | Sign out |
+-----+

+-----+-----+-----+
| RIGHT SIDEBAR (collapsible) | MAIN CHAT | MEMBERS / CONTEXT |
| (rooms + contacts) | | |
+-----+-----+-----+
| Search [ ] | # engineering-room | Room info |
| | "Classic room description here..." | Public room |
| ROOMS | Owner: alice |
| > Public Rooms | [10:21] Bob: Hello team | Admins |
| • general (3) | [10:22] Alice: Uploading spec | - alice |
| • engineering | [10:23] You: Here's the file | - dave |
| • random | | |
| | spec-v3.pdf | Members (38) |
| > Private Rooms | comment: latest requirements | • Alice |
| • core-team (1) | | • Bob |
| • ops | | ● Carol (AFK) |
| | [10:25] Carol replied to Bob: | ○ Mike (offline) |
| CONTACTS | > Hello team | |
| • Alice | Can we make this private? | [Invite user] |
| ● Bob | | [Manage room] |
| ○ Carol (2) | ----- older messages ----- |
| | |
| [Create room] | |
+-----+-----+-----+
| [😊] [Attach] [Replying to: Bob ×] | [ message input ] |
| | [ multiline text... ] |
| | [ Send ] |

```

+-----+

+-----+
| Manage Room: #engineering-room [X] |
+-----+

| Tabs: [Members] [Admins] [Banned users] [Invitations] [Settings] |
+-----+

[Members]

+-----+
| Search member [_____] |
+-----+
| Username Status Role Actions |
| alice online Owner -- |
| dave AFK Admin [Remove admin] [Ban] |
| bob online Member [Make admin] [Ban] [Remove from room] |
| carol offline Member [Make admin] [Ban] [Remove from room] |
+-----+

[Admins]

+-----+
| Current admins: alice, dave |
| alice = owner (cannot lose admin rights) |
| dave [Remove admin] |
+-----+

[Banned users]

+-----+
| Username Banned by Date/time Actions |
| mike alice 2026-04-18 13:25 [Unban] |
| eve dave 2026-04-18 13:40 [Unban] |
+-----+

[Invitations]

```
+-----+
| Invite by username [ _____ ] [Send invite] |
+-----+
```

[Settings]

```
+-----+
| Room name      [ engineering-room ] |
| Description    [ backend + frontend discussions ] |
| Visibility     (•) Public  ( ) Private |
|               |               |
| [ Save changes ]               [ Delete room ] |
+-----+
```